

PROYECTO

SUPPLY-CORE

Sistema inteligente de abastecimiento, inventario y compras

Documento de introducción técnica y funcional para el equipo de desarrollo

Proyecto formativo — Ingeniería de Desarrollo de Software

Versión 1.0

Contenido

1. ¿Qué es Supply-Core?	3
2. El problema que resolvemos	3
2.1 Propuesta de valor	3
3. Alcance de la primera versión	4
3.1 Sí entra en esta versión	4
3.2 No entra en esta versión (y por qué)	5
4. Actores y roles	5
5. Casos de uso	6
5.1 Especificación de los casos de uso	6
6. El flujo completo del negocio, paso a paso	12
7. Módulos del sistema	13
8. Cómo funciona la inteligencia artificial (el diferencial del proyecto)	14
8.1 Qué sí hace la IA	14
8.2 Qué nunca hace la IA	14
8.3 Flujo técnico	14
8.4 Ejemplo de comparación	15
9. Arquitectura y tecnologías	15
9.1 Estructura de paquetes del backend (monolito modular)	16
9.2 Capas dentro del backend	16
9.3 Seguridad no negociable	16
10. Modelo de datos	17
10.1 Resumen de las correcciones aplicadas al MER original	17
11. Roadmap de desarrollo	18
12. Checklist antes de empezar a programar	18
Anexo A. Script de base de datos (MySQL 8)	19

1. ¿Qué es Supply-Core?

Supply-Core es el software a la medida que vamos a construir como proyecto formativo. Es una plataforma web que centraliza todo el ciclo de abastecimiento de una empresa: inventario, proveedores, solicitudes de compra, cotizaciones, análisis inteligente de esas cotizaciones, órdenes de compra, pagos registrados y recepción de mercancía.

Lo que lo diferencia de un ERP de compras genérico es su módulo de inteligencia artificial: cuando varios proveedores cotizan el mismo producto, el sistema no se limita a listar precios. Extrae automáticamente los datos de cada cotización (así vengan en Excel, PDF o imagen), los normaliza a una unidad comparable, calcula un puntaje ponderado por precio, calidad, tiempo de entrega, garantía e historial, y presenta un ranking de la mejor a la peor alternativa, explicando el porqué de cada posición.

Idea central del proyecto

El sistema no debe limitarse a decir “quién cotizó más barato”. Debe responder “cuál alternativa es más conveniente y por qué”, usando evidencia, reglas de negocio y datos históricos. La IA analiza y explica; el software calcula y controla; la persona autorizada decide.

Este documento explica, en orden, qué problema resolvemos, cómo funciona el flujo completo del negocio, con qué tecnologías lo vamos a construir y cómo está diseñada la base de datos. La idea es que cualquiera del equipo pueda leerlo de principio a fin y quedar en capacidad de programar su módulo sin necesitar una explicación adicional.

2. El problema que resolvemos

Hoy, en una empresa sin un sistema como este, el proceso de comprar algo está fragmentado: el inventario se lleva en un Excel, las cotizaciones llegan por correo o WhatsApp en formatos distintos, la comparación entre proveedores se hace a mano copiando precios en otra hoja, y cuando llega la mercancía nadie puede reconstruir fácilmente por qué se eligió ese proveedor ni si lo recibido coincide con lo pedido.

Esa fragmentación genera tres problemas concretos que Supply-Core ataca directamente:

- No hay trazabilidad: nadie puede reconstruir con evidencia por qué se decidió comprarle a un proveedor y no a otro.
- La comparación es manual y lenta: normalizar “caja de 24 unidades” contra “docena” contra “50 unidades sueltas” a mano es tedioso y propenso a error.
- El inventario vive separado de las compras: nadie sabe con certeza, en tiempo real, cuánto stock hay ni cuándo se debería generar una nueva solicitud.

2.1 Propuesta de valor

Antes	Con Supply-Core
Cotizaciones en correos y archivos sueltos	Cotizaciones centralizadas en un proceso RFQ único
Comparación manual, precio como único criterio	Normalización automática + score ponderado (precio, calidad, entrega, garantía, historial, riesgo)

Antes	Con Supply-Core
Decisiones sin registro del motivo	Recomendación explicable por IA + decisión humana registrada con motivo
Stock llevado a mano, desactualizado	Stock por bodega, actualizado únicamente por movimientos controlados
Recepción y compra desconectadas	Recepción vinculada a la orden; el stock solo sube cuando el almacén confirma lo recibido

3. Alcance de la primera versión

Este es el punto más importante para que todo el equipo trabaje alineado: qué construimos ahora y qué se deja explícitamente para después. Un proyecto formativo se atrasa cuando el equipo empieza a construir funcionalidades “porque quedan bien” y no porque el alcance las pida.

Regla de oro del proyecto

No se construyen funcionalidades por entusiasmo tecnológico. Cada funcionalidad debe responder a una necesidad real del proceso de abastecimiento, tener un responsable, una regla de negocio, un estado y un criterio de aceptación verificable.

3.1 Sí entra en esta versión

- Autenticación, roles y permisos.
- Configuración de empresa, sedes, centros de costo, categorías, unidades de medida.
- Productos e inventario: catálogo, stock por bodega, movimientos, mínimos/máximos.
- Proveedores y portal propio del proveedor.
- Solicitudes de compra, con aprobación.
- RFQ (solicitud de cotización) e invitación a proveedores.
- Cotizaciones: formulario o carga de archivo (Excel, PDF, imagen).
- IA y comparación inteligente: extracción, normalización, score, ranking, explicación y detección de anomalías.
- Adjudicación y orden de compra.
- Registro de pago (sin pasarela de pagos ni contabilidad).
- Recepción de mercancía y actualización de inventario.
- Evaluación de proveedores.
- Dashboards y reportes.
- Auditoría de acciones sensibles.

3.2 No entra en esta versión (y por qué)

Tema	Por qué se deja para después
Módulo de ventas	No forma parte del alcance actual; se documenta la base de datos de forma que se pueda agregar después sin rediseñar todo.
Multiempresa real / SaaS	Es un software a la medida para una sola empresa; el código queda modular mas no se construye la operación multiempresa ahora.
Marketplace abierto	Requiere reputación, moderación y un modelo comercial que no aporta al objetivo del proyecto formativo.
Pasarela de pagos propia	Solo se registra el pago que la empresa ya hizo por otro medio; no se maneja dinero dentro del sistema.
Facturación electrónica / DIAN	Es un dominio tributario aparte, fuera del alcance académico.
Microservicios	Se construye un monolito modular; migrar a microservicios sería una evolución posterior, no el punto de partida.
Machine learning propio / agentes autónomos	Primero necesitamos datos históricos limpios; la IA de esta versión usa un modelo ya entrenado (API), no entrenamos modelos propios.

4. Actores y roles

Antes de tocar código hay que tener claro quién hace qué. Cada pantalla y cada botón del sistema existen para uno de estos roles; si una funcionalidad no tiene un rol claro detrás, probablemente no debería construirse todavía.

Rol	Qué hace
Administrador del sistema	Configura usuarios, roles, seguridad y catálogos. No decide compras.
Administrador de compras	Define políticas de compra, categorías y proveedores habilitados.
Solicitante	Registra una necesidad de compra y hace seguimiento a su solicitud.
Comprador / responsable de cotizaciones	Crea la RFQ, invita proveedores, revisa cotizaciones, ejecuta el análisis y gestiona la orden.
Aprobador / jefe de área	Aprueba, rechaza o devuelve solicitudes y adjudicaciones según su límite de autorización.
Almacenista	Recibe productos, registra cantidades, diferencias y movimientos de inventario.
Responsable administrativo	Registra el pago de la compra y adjunta el comprobante.
Proveedor	Administra sus datos, ve solicitudes, presenta cotizaciones, confirma órdenes y entrega.
Auditor / gerencia	Consulta trazabilidad, indicadores y desempeño, sin modificar operaciones.

5. Casos de uso

Los actores de la sección anterior interactúan con Supply-Core a través de 16 casos de uso. Cada uno tiene un actor principal, una precondition, un flujo normal, al menos un flujo alternativo y una postcondición verificable — así cualquiera del equipo puede tomar un caso de uso y programarlo sin depender de una explicación adicional.

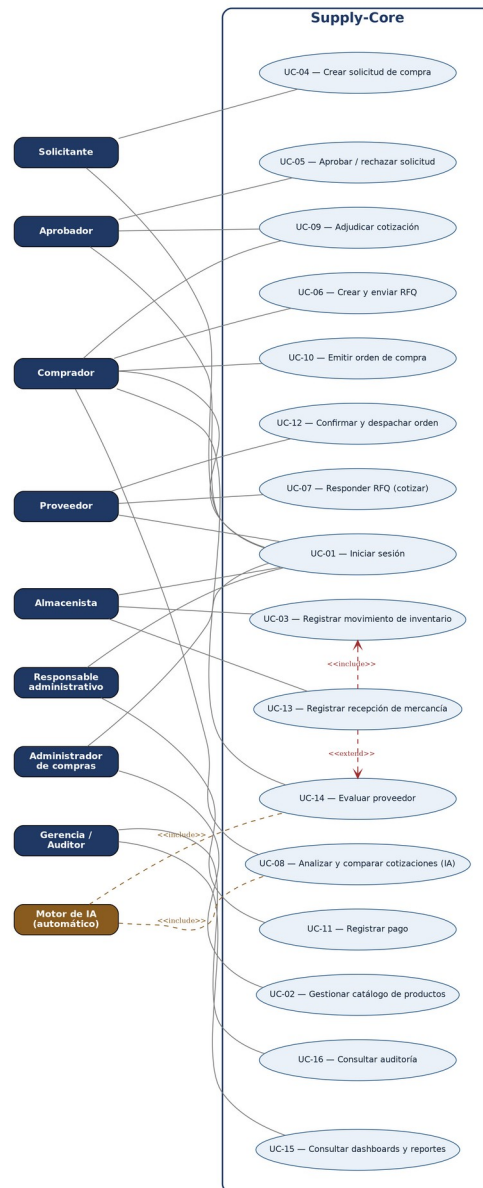


Figura 2. Diagrama de casos de uso de Supply-Core.

5.1 Especificación de los casos de uso

UC-01 — Iniciar sesión

Actor principal	Todos los roles
Precondición	El usuario tiene una cuenta activa en el sistema.

Flujo principal	1. El usuario ingresa correo y contraseña. 2. El sistema valida las credenciales. 3. El sistema carga el menú según los roles y permisos del usuario.
Flujo alternativo / excepciones	• Credenciales inválidas: se rechaza el acceso sin indicar cuál dato falló. • Cuenta inactiva: se rechaza el acceso y se informa que debe contactar al administrador.
Postcondición	El usuario queda autenticado con una sesión activa.

UC-02 — Gestionar catálogo de productos

Actor principal	Administrador de compras
Precondición	El usuario tiene permiso sobre el módulo de catálogo.
Flujo principal	1. Registra o edita un producto (código, nombre, categoría, unidad, stock mínimo/máximo). 2. El sistema valida que el código no esté duplicado. 3. El sistema guarda el producto.
Flujo alternativo / excepciones	• Código duplicado: el sistema lo rechaza y muestra el registro existente. • Desactivar producto: se impide su uso en nuevas operaciones sin borrar su historial.
Postcondición	El catálogo queda actualizado y disponible para solicitudes, RFQ y recepción.

UC-03 — Registrar movimiento de inventario

Actor principal	Almacenista
Precondición	El producto y la bodega existen y están activos.
Flujo principal	1. El usuario selecciona producto, bodega y tipo de movimiento (entrada, salida, ajuste, traslado). 2. El sistema valida cantidades y motivo cuando aplica. 3. El sistema registra el movimiento y recalcula el stock de esa bodega.
Flujo alternativo / excepciones	• Ajuste sin motivo: el sistema exige un motivo obligatorio antes de guardar. • Traslado: el sistema exige bodega de origen y destino distintas.
Postcondición	El stock por bodega queda actualizado y el movimiento queda en el historial.

UC-04 — Crear solicitud de compra

Actor principal	Solicitante
------------------------	-------------

Precondición	El usuario está autenticado y tiene un centro de costo asignado.
Flujo principal	1. El usuario registra producto, cantidad, unidad, fecha requerida y justificación. 2. El sistema valida la política de aprobación aplicable. 3. El sistema crea la solicitud en estado pendiente o la envía a aprobación.
Flujo alternativo / excepciones	• Cantidad menor o igual a cero: el sistema no permite continuar.
Postcondición	La solicitud queda creada y visible para el solicitante y el aprobador.

UC-05 — Aprobar / rechazar solicitud

Actor principal	Aprobador
Precondición	Existe una solicitud pendiente de aprobación asignada al usuario.
Flujo principal	1. El aprobador revisa la solicitud. 2. Decide aprobar, rechazar o devolver, dejando observaciones. 3. El sistema registra la decisión y notifica al solicitante.
Flujo alternativo / excepciones	• Devolución: la solicitud vuelve al solicitante para completar información.
Postcondición	La solicitud queda en estado Aprobada, Rechazada o Devuelta, con auditoría de la decisión.

UC-06 — Crear y enviar RFQ

Actor principal	Comprador
Precondición	Existe al menos una solicitud de compra aprobada.
Flujo principal	1. El comprador define productos, cantidades, condiciones y fecha límite. 2. Selecciona los proveedores a invitar. 3. El sistema envía la invitación a cada proveedor y abre la RFQ.
Flujo alternativo / excepciones	• Ningún proveedor habilitado para la categoría: el sistema alerta antes de enviar.
Postcondición	La RFQ queda abierta y visible para los proveedores invitados.

UC-07 — Responder RFQ (cotizar)

Actor principal	Proveedor
------------------------	-----------

Precondición	El proveedor fue invitado a una RFQ vigente (no cerrada).
Flujo principal	1. El proveedor abre la RFQ desde su portal. 2. Diligencia el formulario o sube un archivo (Excel, PDF, imagen). 3. El sistema conserva el archivo original y crea la cotización.
Flujo alternativo / excepciones	• RFQ cerrada o vencida: el sistema no permite cotizar.
Postcondición	La cotización queda registrada en estado Enviada, lista para el análisis de IA.

UC-08 — Analizar y comparar cotizaciones (IA)

Actor principal	Comprador
Precondición	Existen una o más cotizaciones enviadas para la misma RFQ.
Flujo principal	1. La IA extrae y normaliza los datos de cada cotización. 2. El backend calcula costo unitario, costo total y score por criterio. 3. El sistema publica el ranking de mejor a peor alternativa con la explicación de la IA.
Flujo alternativo / excepciones	• Confianza baja en un dato crítico: el sistema obliga al comprador a confirmarlo antes de publicar el ranking. • Precio atípico: se marca como alerta, sin excluir la oferta automáticamente.
Postcondición	El ranking queda disponible para adjudicación, con el detalle de puntajes conservado.

UC-09 — Adjudicar cotización

Actor principal	Comprador / Aprobador
Precondición	Existe un análisis de IA publicado para la RFQ.
Flujo principal	1. El comprador revisa el ranking y elige una cotización. 2. Si se aparta de la recomendación, registra un motivo obligatorio. 3. Si el monto o la política lo exige, un aprobador valida la adjudicación.
Flujo alternativo / excepciones	• Monto supera el límite del comprador: pasa a aprobación adicional antes de continuar.
Postcondición	Queda una adjudicación registrada con proveedor, motivo y usuario responsable.

UC-10 — Emitir orden de compra

Actor principal	Comprador
------------------------	-----------

Precondición	Existe una adjudicación aprobada.
Flujo principal	1. El sistema genera la orden con proveedor, productos, cantidades, precios y lugar de entrega. 2. El comprador confirma la emisión. 3. El sistema notifica al proveedor.
Flujo alternativo / excepciones	• Modificación posterior: no se sobrescribe la orden; se crea una nueva versión con motivo.
Postcondición	La orden queda emitida, inmutable en sus datos esenciales, y vinculada a la adjudicación.

UC-11 — Registrar pago

Actor principal	Responsable administrativo
Precondición	Existe una orden de compra emitida.
Flujo principal	1. El usuario registra método, valor, fecha y referencia del pago. 2. Adjunta el comprobante si está disponible. 3. El sistema guarda el registro en estado Registrado.
Flujo alternativo / excepciones	• Sin comprobante disponible: el pago queda registrado igual, marcado para adjuntar evidencia después.
Postcondición	El pago queda con estado explícito (Pendiente/Registrado) vinculado a la orden.

UC-12 — Confirmar y despachar orden

Actor principal	Proveedor
Precondición	La orden fue emitida y notificada al proveedor.
Flujo principal	1. El proveedor confirma la aceptación de la orden. 2. Actualiza el estado a medida que prepara y despacha. 3. Puede adjuntar evidencia de despacho.
Flujo alternativo / excepciones	• Proveedor rechaza la orden: el comprador es notificado para gestionar una alternativa.
Postcondición	La orden queda en estado Despachada, con evidencia asociada si existe.

UC-13 — Registrar recepción de mercancía

Actor principal	Almacenista
Precondición	La orden está despachada o en tránsito.
Flujo principal	1. El almacenista registra cantidad recibida por línea. 2. Registra diferencias (faltante, rechazado) y evidencia fotográfica.

	3. El sistema genera el movimiento de inventario correspondiente.
Flujo alterno / excepciones	• Recepción parcial: el saldo pendiente queda abierto para una recepción posterior. • Exceso recibido: requiere revisión manual, no se acepta automáticamente.
Postcondición	El inventario se actualiza únicamente con lo efectivamente recibido; la orden avanza de estado.

UC-14 — Evaluar proveedor

Actor principal	Sistema (automático) / Comprador
Precondición	Se cerró una recepción u orden asociada al proveedor.
Flujo principal	1. El sistema calcula puntualidad, cumplimiento, incidencias y calidad de la operación. 2. Actualiza el score acumulado del proveedor. 3. El comprador puede consultar el historial actualizado.
Flujo alterno / excepciones	• Datos insuficientes para un indicador: ese indicador queda sin calcular en vez de asumir un valor por defecto.
Postcondición	El score del proveedor queda actualizado y disponible para futuras adjudicaciones.

UC-15 — Consultar dashboards y reportes

Actor principal	Gerencia / Auditor
Precondición	El usuario tiene permiso de consulta sobre reportes.
Flujo principal	1. El usuario selecciona el dashboard (gasto, inventario, proveedores). 2. El sistema calcula los indicadores a partir de datos transaccionales. 3. El usuario filtra por fecha, categoría o proveedor.
Flujo alterno / excepciones	• Sin datos en el período seleccionado: el sistema muestra el estado vacío, no un error.
Postcondición	El usuario obtiene indicadores actualizados sin modificar ninguna operación.

UC-16 — Consultar auditoría

Actor principal	Auditor
Precondición	El usuario tiene permiso de auditoría.
Flujo principal	1. El usuario busca por usuario, entidad, acción o rango de fechas. 2. El sistema muestra el historial correspondiente con

	resultado y contexto.
Flujo alterno / excepciones	• Sin permisos de auditoría: el sistema bloquea el acceso al módulo.
Postcondición	El auditor puede reconstruir quién hizo qué y cuándo, sin alterar los registros.

6. El flujo completo del negocio, paso a paso

Este es el flujo de punta a punta que el equipo debe poder demostrar con datos de ejemplo. Cada paso corresponde a una pantalla o un proceso interno concreto; en conjunto forman el caso de uso principal del proyecto.

1. Se detecta una necesidad

El inventario cae por debajo del stock mínimo, o un usuario registra manualmente una necesidad.

2. Se crea la solicitud de compra

Se registra producto, cantidad, unidad, fecha requerida, centro de costo y justificación.

3. Se valida y aprueba

El sistema revisa la política aplicable; si requiere aprobación, el aprobador acepta, rechaza o devuelve la solicitud. Todo queda auditado.

4. Se crea la RFQ

El comprador define qué se pide, la fecha límite, las condiciones y qué proveedores se invitan.

5. El proveedor recibe la RFQ

Desde su portal propio, ve la solicitud y sus fechas límite.

6. El proveedor cotiza

Puede usar un formulario estructurado o subir un archivo (Excel, PDF o imagen). El sistema conserva siempre el archivo original.

7. La IA procesa la cotización

Extrae producto, cantidad, precio, impuestos, descuentos, plazos y garantía, y devuelve un nivel de confianza por cada dato.

8. El backend valida y calcula

Los cálculos monetarios y las reglas de negocio se ejecutan en Java, nunca en el modelo de IA.

9. Se normaliza y compara

El sistema convierte todo a una unidad comparable (por ejemplo, costo por unidad) y calcula el costo total, no solo el precio unitario.

10. La IA explica el ranking

Resume ventajas, desventajas y anomalías de cada oferta, y el sistema muestra de la mejor a la peor alternativa.

11. El comprador decide

Puede seguir la recomendación o elegir otra oferta, dejando un motivo obligatorio si se aparta de la recomendación.

12. Se aprueba la adjudicación

Si el monto o la política lo exige, un aprobador o comité valida la decisión.

13. Se genera la orden de compra

Queda vinculada a la solicitud, la RFQ, la cotización y el proveedor elegido. Una vez emitida, es inmutable: los cambios quedan como una nueva versión.

14. Se registra el pago

Se guarda método, valor, fecha, referencia y comprobante. No hay pasarela de pagos ni contabilidad, solo un registro.

15. El proveedor confirma y entrega

Actualiza el estado de la orden y puede subir evidencia de despacho.

16. El almacén recibe

Registra cantidad esperada, recibida, diferencias, calidad y evidencia fotográfica.

17. El inventario se actualiza

La recepción es el único evento que aumenta el stock. Una orden emitida, por sí sola, nunca mueve inventario.

18. Se evalúa al proveedor

Se actualizan indicadores de puntualidad, cumplimiento, calidad e incidencias.

19. El proceso se cierra

Toda la operación queda consultable y alimenta el historial para futuras comparaciones.

Regla que no se negocia

El stock nunca se edita a mano desde una pantalla. El saldo es siempre consecuencia de un movimiento de inventario (entrada, salida, ajuste, traslado o recepción), y la recepción es el único evento que incrementa el stock por una compra.

7. Módulos del sistema

#	Módulo	Para qué sirve
1	Autenticación y seguridad	Login, roles, permisos, sesiones y auditoría.
2	Configuración de empresa	Sedes, centros de costo, políticas de aprobación.
3	Productos e inventario	Catálogo, stock por bodega, movimientos, mínimos/máximos.
4	Proveedores	Registro, documentos, ubicación, evaluación e historial.
5	Solicitudes de compra	Creación, aprobación y seguimiento.
6	RFQ	Generación de la solicitud de cotización e invitación a proveedores.
7	Cotizaciones	Formulario, carga de archivos, estados.
8	IA y comparación inteligente	Extracción, normalización, score, ranking y explicación.

#	Módulo	Para qué sirve
9	Adjudicación y orden de compra	Selección, aprobación y generación de la orden.
10	Recepción	Cantidades recibidas, evidencias y actualización de inventario.
11	Pagos y documentos de compra	Registro del pago y del comprobante.
12	Reportes y dashboards	Gasto, ahorro, inventario, proveedores y alertas.
13	Auditoría	Historial de acciones sensibles con usuario, fecha y resultado.

8. Cómo funciona la inteligencia artificial (el diferencial del proyecto)

Este es el módulo que hace innovador al proyecto, así que todo el equipo debe entender un principio antes de programarlo: la IA nunca decide ni calcula dinero. La IA lee, entiende y explica; el backend en Java calcula y valida; la persona autorizada decide.

8.1 Qué sí hace la IA

- Extraer datos de PDF, Excel o imagen (producto, cantidad, precio, plazos, garantía).
- Clasificar y normalizar semánticamente descripciones de productos.
- Explicar en lenguaje natural por qué una oferta quedó primera, segunda o tercera.
- Señalar anomalías (por ejemplo, un precio muy alejado del histórico) como una alerta, nunca como un bloqueo automático.

8.2 Qué nunca hace la IA

- Calcular precios, subtotales, impuestos o el score final: eso lo hace el backend con una fórmula determinística.
- Decidir el proveedor ganador: la decisión final siempre queda registrada a nombre de una persona autorizada.

8.3 Flujo técnico

- El proveedor sube el documento de la cotización.
- El backend guarda el archivo y crea un registro de procesamiento.
- Se dispara un proceso asíncrono (cola/job) para no bloquear la interfaz mientras la IA trabaja.
- El servicio de IA recibe el documento y un esquema de salida estructurado (JSON), usando function calling / structured outputs para forzar una respuesta validable.
- La respuesta se valida contra ese esquema; cada campo trae un nivel de confianza.
- Si un dato crítico (cantidad, precio, unidad) tiene confianza baja, el sistema obliga al comprador a confirmarlo antes de seguir.
- El backend calcula el costo unitario, el costo total y el score ponderado por criterio (precio, calidad, entrega, garantía, historial, riesgo).

- El resultado se guarda junto con la explicación de la IA y se publica el ranking para decisión humana.

8.4 Ejemplo de comparación

Criterio	Proveedor A	Proveedor B	Proveedor C
Precio unitario	\$1.200	\$1.100	\$1.140
Calidad	96	80	92
Entrega	95	70	90
Garantía	90	80	95
Score final	93,4	80,8	91,5
Resultado	1°	3°	2°

Por qué esto importa para la sustentación

Un jurado o un cliente va a preguntar “¿y si la IA se equivoca?”. La respuesta correcta del proyecto es: la IA no calcula dinero ni decide sola, así que un error de la IA nunca puede traducirse en una compra mal hecha sin que una persona lo haya revisado y aprobado.

9. Arquitectura y tecnologías

Para un proyecto formativo se recomienda un monolito modular, no microservicios desde el día uno: mantiene los límites de dominio claros sin la complejidad operativa de desplegar y observar varios servicios independientes. Esta es la pila tecnológica recomendada y el motivo de cada elección.

Capa	Tecnología	Motivo
Frontend	Angular	Aplicación administrativa robusta: formularios reactivos, guards de rutas por rol, y buen soporte para dashboards con muchos componentes.
Backend	Java 21 + Spring Boot	Dominio fuerte, validación, seguridad y API REST con buena mantenibilidad para un equipo académico.
Base de datos	MySQL	Persistencia transaccional del proyecto; buen soporte, documentación y compatibilidad con Flyway.
Documentos	Object Storage (archivos)	Los PDF, Excel e imágenes de cotizaciones no deben vivir dentro de la base de datos.
Procesos asíncronos	Cola / jobs ligeros	Procesar la IA y los documentos sin bloquear la interfaz del usuario.
Inteligencia artificial	API de un proveedor de modelos, encapsulada detrás de una interfaz propia	El resto del sistema no depende directamente del proveedor de IA elegido; se puede cambiar sin tocar el dominio.

Capa	Tecnología	Motivo
Mapas	Google Maps Platform	Geocodificación de la dirección de proveedores (server-to-server, nunca exponiendo la llave en el navegador).
Documentación de API	OpenAPI / Swagger	Contrato claro y prueba de endpoints entre frontend y backend.
Migraciones de base de datos	Flyway	Versionado controlado del esquema, cada módulo con sus propias migraciones.

9.1 Estructura de paquetes del backend (monolito modular)

El backend se organiza por dominio, no por capa técnica, para que cada módulo del negocio tenga sus límites claros:

```
src/main/java/...
├─ security
├─ usuario
├─ empresa
├─ catalogo
├─ inventario
├─ proveedor
├─ compra
│   ├─ solicitud
│   ├─ rfq
│   ├─ cotizacion
│   ├─ analisis
│   ├─ adjudicacion
│   └─ orden
├─ recepcion
├─ comprobante_compra
├─ pago
├─ documento
├─ notificacion
├─ auditoria
└─ ia
```

9.2 Capas dentro del backend

```
Controller -> Service -> Reglas de dominio -> Repository
    |
    DTOs / Mappers
    |
    Persistencia / MySQL
```

Integraciones externas siempre detrás de una interfaz propia:

```
IAClient | MapsClient | EmailClient
```

9.3 Seguridad no negociable

- Contraseñas siempre con hash fuerte, nunca en texto plano.

- Autorización por rol y permiso, con segregación de funciones (quien solicita no puede ser quien aprueba y registra el pago a la vez, si la política lo exige).
- Toda acción sensible queda en la auditoría: usuario, acción, entidad afectada y fecha.
- Las claves de IA y de Google Maps viven solo en variables de entorno del backend, nunca en el frontend ni en el repositorio.

10. Modelo de datos

El modelo se construyó a partir del MER inicial del proyecto, con siete correcciones aplicadas para que el esquema soporte de verdad las reglas que el negocio exige: stock por bodega (no solo un número global), un catálogo normalizado de criterios de evaluación, una copia congelada (“snapshot”) de los pesos usados en cada análisis para que el histórico nunca cambie retroactivamente, los siete indicadores de desempeño del proveedor, un estado explícito para el pago, la relación real de muchos-a-muchos entre usuario y rol, y una llave polimórfica real en la tabla de documentos.

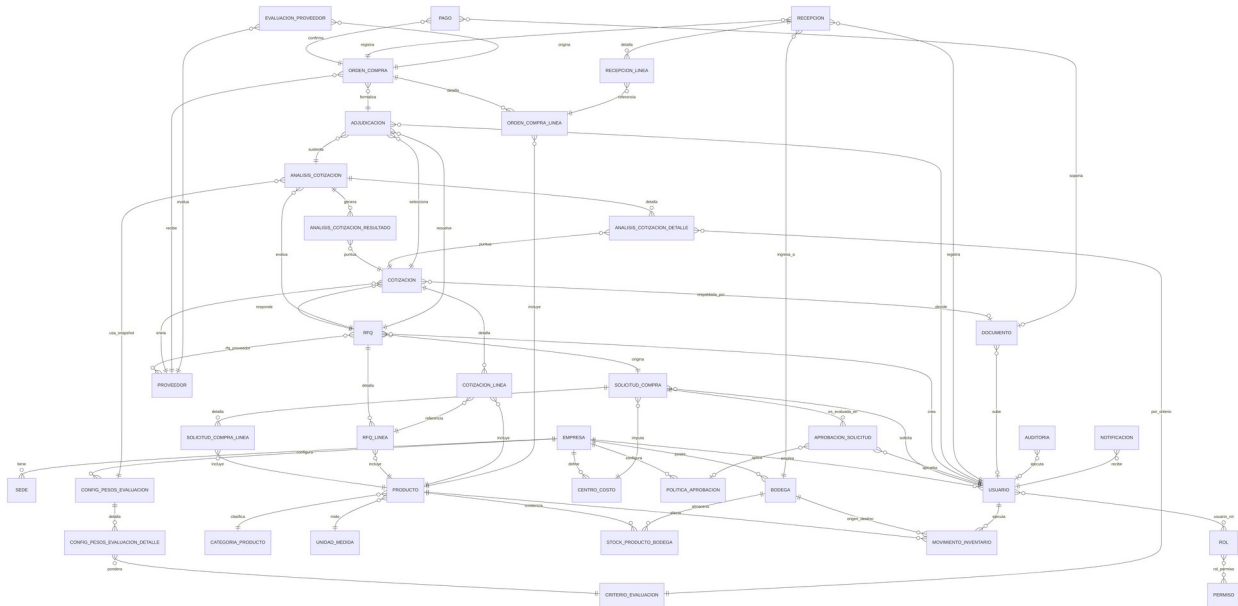


Figura 1. Modelo entidad-relación de Supply-Core.

10.1 Resumen de las correcciones aplicadas al MER original

- BODEGA + STOCK_PRODUCTO_BODEGA: la existencia real del inventario vive por bodega, no como un número único en el producto; permite traslados entre bodegas.
- CRITERIO_EVALUACION: normaliza los criterios (precio, calidad, entrega, garantía, historial, riesgo, condiciones) en vez de columnas fijas que se desactualizan.
- CONFIG_PESOS_EVALUACION versionada: cada análisis de IA guarda qué versión de los pesos usó, así después no se pueda “reescribir la historia” cambiando la matriz de pesos.
- EVALUACION_PROVEEDOR con los siete indicadores reales (puntualidad, cumplimiento, incidencias, calidad, precio, respuesta y score).
- PAGO.estado (PENDIENTE / REGISTRADO), que faltaba en el modelo original.
- USUARIO_ROL como tabla de muchos-a-muchos, porque un usuario puede tener más de un rol.

- DOCUMENTO con entidad_tipo + entidad_id como llave polimórfica real, en vez de relaciones implícitas sin tipar.

El script completo de creación de las tablas está en el Anexo A de este documento, listo para ejecutarse en MySQL 8.

11. Roadmap de desarrollo

Este es el orden recomendado de construcción. No es opcional: cada fase depende de que la anterior ya exista, y saltarse el orden es la causa más común de retrabajo en un proyecto de este tamaño.

Fase	Contenido
0	Cerrar reglas de negocio y casos de uso; crear el modelo entidad-relación (este documento).
1	Migraciones de base de datos y autenticación/autorización.
2	Productos e inventario (catálogo, bodegas, movimientos).
3	Proveedores y portal del proveedor.
4	Solicitudes de compra y RFQ.
5	Cotizaciones (formulario y carga de archivos).
6	IA y motor de comparación (extracción, normalización, score, ranking, explicación).
7	Adjudicación, orden de compra, registro de pago y recepción (actualiza inventario).
8	Dashboards y auditoría.
9	Pruebas end-to-end, seguridad, documentación y despliegue.

12. Checklist antes de empezar a programar

- Los roles y permisos están aprobados por todo el equipo.
- Los estados y transiciones de cada proceso están definidos (solicitud, RFQ, cotización, orden).
- Las entidades principales y sus relaciones están acordadas (este documento + el script del Anexo A).
- El flujo completo (sección 6) se puede contar sin contradicciones, de memoria, por cualquiera del equipo.
- Todos entienden que la IA explica y el backend calcula — nunca al revés.
- La recepción es el único evento que aumenta el inventario por compras.
- Cada quien sabe en qué módulo va a trabajar primero, según el roadmap de la sección 11.

Anexo A. Script de base de datos (MySQL 8)

Script completo y ejecutable de creación del esquema, con las siete correcciones descritas en la sección 10.1 ya aplicadas y comentadas en el propio código.

```
-- =====
-- Sistema de Abastecimiento (Inventario + Compras + Proveedores
-- + RFQ/Cotizaciones + IA + Adjudicación + Órdenes + Pago +
-- Recepción)
-- MySQL 8.x – modelo corregido a partir del MER original.
--
-- Correcciones aplicadas respecto al MER entregado:
-- [1] bodega + stock_producto_bodega (stock por ubicación,
--     soporta traslados)
-- [2] criterios de evaluación normalizados
--     (config_pesos_evaluacion_detalle / analisis_cotizacion_detalle)
--     en vez de columnas fijas peso_x / puntaje_x
-- [3] snapshot de pesos: cada análisis referencia una versión
--     congelada de config_pesos_evaluacion (vigente_desde/hasta)
-- [4] evaluacion_proveedor con los 7 indicadores de la sección 18
--     + score_actual como rollup en proveedor
-- [5] pago.estado (PENDIENTE/REGISTRADO)
-- [6] usuario_rol como tabla N:M (un usuario puede tener varios roles)
-- [7] documento con entidad_tipo/entidad_id (FK polimórfica real)
-- =====

SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

-- =====
-- NÚCLEO Y SEGURIDAD
-- =====

CREATE TABLE empresa (
  id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  razon_social      VARCHAR(200)    NOT NULL,
  nit               VARCHAR(30)     NOT NULL UNIQUE,
  direccion_principal VARCHAR(250),
  moneda_base       CHAR(3)         NOT NULL DEFAULT 'COP',
  creado_en         DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE sede (
  id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  empresa_id        BIGINT UNSIGNED NOT NULL,
  nombre            VARCHAR(150) NOT NULL,
  direccion          VARCHAR(250),
  latitud            DECIMAL(10,7),
  longitud           DECIMAL(10,7),
  activo            TINYINT(1) NOT NULL DEFAULT 1,
  FOREIGN KEY (empresa_id) REFERENCES empresa(id)
) ENGINE=InnoDB;

CREATE TABLE rol (
  id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  codigo            VARCHAR(50) NOT NULL UNIQUE,
```

```

    nombre      VARCHAR(100) NOT NULL,
    descripcion VARCHAR(250)
) ENGINE=InnoDB;

CREATE TABLE permiso (
    id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    codigo VARCHAR(80) NOT NULL UNIQUE,
    nombre VARCHAR(150) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE rol_permiso (
    rol_id      BIGINT UNSIGNED NOT NULL,
    permiso_id BIGINT UNSIGNED NOT NULL,
    PRIMARY KEY (rol_id, permiso_id),
    FOREIGN KEY (rol_id) REFERENCES rol(id),
    FOREIGN KEY (permiso_id) REFERENCES permiso(id)
) ENGINE=InnoDB;

CREATE TABLE usuario (
    id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    empresa_id  BIGINT UNSIGNED NOT NULL,
    nombre_completo VARCHAR(200) NOT NULL,
    correo      VARCHAR(150) NOT NULL UNIQUE,
    hash_password VARCHAR(255) NOT NULL,
    activo      TINYINT(1) NOT NULL DEFAULT 1,
    ultimo_acceso DATETIME NULL,
    FOREIGN KEY (empresa_id) REFERENCES empresa(id)
) ENGINE=InnoDB;

-- [6] N:M real usuario-rol (un usuario puede tener varios roles)
CREATE TABLE usuario_rol (
    usuario_id BIGINT UNSIGNED NOT NULL,
    rol_id     BIGINT UNSIGNED NOT NULL,
    PRIMARY KEY (usuario_id, rol_id),
    FOREIGN KEY (usuario_id) REFERENCES usuario(id),
    FOREIGN KEY (rol_id) REFERENCES rol(id)
) ENGINE=InnoDB;

-- =====
-- M02: CONFIGURACIÓN Y MAESTROS
-- =====

CREATE TABLE centro_costo (
    id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    codigo VARCHAR(30) NOT NULL UNIQUE,
    nombre VARCHAR(150) NOT NULL,
    activo TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB;

CREATE TABLE categoria_producto (
    id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE unidad_medida (
    id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    codigo      VARCHAR(20) NOT NULL UNIQUE,

```

```

    nombre          VARCHAR(80) NOT NULL,
    factor_conversion_base DECIMAL(18,6) NOT NULL DEFAULT 1
) ENGINE=InnoDB;

CREATE TABLE politica_aprobacion (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    empresa_id     BIGINT UNSIGNED NOT NULL,
    monto_desde    DECIMAL(18,2) NOT NULL,
    monto_hasta    DECIMAL(18,2) NOT NULL,
    activo         TINYINT(1) NOT NULL DEFAULT 1,
    FOREIGN KEY (empresa_id) REFERENCES empresa(id)
) ENGINE=InnoDB;

-- [2] catálogo de criterios (precio, calidad, entrega, garantía,
--    historial, riesgo, condiciones...) en vez de columnas fijas
CREATE TABLE criterio_evaluacion (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    codigo VARCHAR(40) NOT NULL UNIQUE,
    nombre VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

-- [3] cabecera versionada de la matriz de pesos
CREATE TABLE config_pesos_evaluacion (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    empresa_id     BIGINT UNSIGNED NOT NULL,
    vigente_desde  DATETIME NOT NULL,
    vigente_hasta  DATETIME NULL,
    activo         TINYINT(1) NOT NULL DEFAULT 1,
    FOREIGN KEY (empresa_id) REFERENCES empresa(id)
) ENGINE=InnoDB;

-- [2][3] detalle: peso por criterio para esa versión de la matriz.
-- Un CHECK a nivel app debe validar que SUM(peso) = 100 por config.
CREATE TABLE config_pesos_evaluacion_detalle (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    config_pesos_evaluacion_id BIGINT UNSIGNED NOT NULL,
    criterio_evaluacion_id BIGINT UNSIGNED NOT NULL,
    peso            DECIMAL(5,2) NOT NULL,
    UNIQUE KEY uq_config_criterio (config_pesos_evaluacion_id, criterio_evaluacion_id),
    FOREIGN KEY (config_pesos_evaluacion_id) REFERENCES config_pesos_evaluacion(id),
    FOREIGN KEY (criterio_evaluacion_id) REFERENCES criterio_evaluacion(id)
) ENGINE=InnoDB;

-- =====
-- M03: PRODUCTOS E INVENTARIO
-- =====

CREATE TABLE producto (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    categoria_producto_id BIGINT UNSIGNED NOT NULL,
    unidad_medida_id BIGINT UNSIGNED NOT NULL,
    codigo          VARCHAR(50) NOT NULL UNIQUE,
    nombre          VARCHAR(200) NOT NULL,
    descripcion     VARCHAR(500),
    stock_minimo    DECIMAL(18,4) NOT NULL DEFAULT 0,
    stock_maximo    DECIMAL(18,4) NULL,
    costo_referencia DECIMAL(18,4) NULL,

```

```

    activo          TINYINT(1) NOT NULL DEFAULT 1,
    FOREIGN KEY (categoria_producto_id) REFERENCES categoria_producto(id),
    FOREIGN KEY (unidad_medida_id) REFERENCES unidad_medida(id)
) ENGINE=InnoDB;

-- [1] bodega/ubicación física de stock
CREATE TABLE bodega (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    empresa_id     BIGINT UNSIGNED NOT NULL,
    nombre         VARCHAR(150) NOT NULL,
    direccion      VARCHAR(250),
    activo         TINYINT(1) NOT NULL DEFAULT 1,
    FOREIGN KEY (empresa_id) REFERENCES empresa(id)
) ENGINE=InnoDB;

-- [1] existencia real: la fuente de verdad del stock es esta tabla,
-- actualizada únicamente por movimiento_inventario (nunca por UPDATE directo)
CREATE TABLE stock_producto_bodega (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    producto_id     BIGINT UNSIGNED NOT NULL,
    bodega_id       BIGINT UNSIGNED NOT NULL,
    cantidad        DECIMAL(18,4) NOT NULL DEFAULT 0,
    UNIQUE KEY uq_producto_bodega (producto_id, bodega_id),
    FOREIGN KEY (producto_id) REFERENCES producto(id),
    FOREIGN KEY (bodega_id) REFERENCES bodega(id)
) ENGINE=InnoDB;

-- [1] soporta traslado (bodega_origen + bodega_destino) además de
-- entrada/salida/ajuste/devolución
CREATE TABLE movimiento_inventario (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    producto_id     BIGINT UNSIGNED NOT NULL,
    bodega_origen_id BIGINT UNSIGNED NULL,
    bodega_destino_id BIGINT UNSIGNED NULL,
    usuario_id      BIGINT UNSIGNED NOT NULL,
    tipo            ENUM('ENTRADA_COMPRA', 'SALIDA_CONSUMO', 'AJUSTE_POSITIVO',
                        'AJUSTE_NEGATIVO', 'DEVOLUCION_PROVEEDOR',
                        'RECEPCION_PARCIAL', 'TRASLADO') NOT NULL,
    cantidad        DECIMAL(18,4) NOT NULL,
    stock_resultante DECIMAL(18,4) NOT NULL,
    referencia_tipo  VARCHAR(50) NULL,
    referencia_id    BIGINT UNSIGNED NULL,
    motivo          VARCHAR(250) NULL,
    creado_en       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (producto_id) REFERENCES producto(id),
    FOREIGN KEY (bodega_origen_id) REFERENCES bodega(id),
    FOREIGN KEY (bodega_destino_id) REFERENCES bodega(id),
    FOREIGN KEY (usuario_id) REFERENCES usuario(id)
) ENGINE=InnoDB;

-- =====
-- M04: PROVEEDORES
-- =====

CREATE TABLE proveedor (
    id              BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    razon_social    VARCHAR(200) NOT NULL,

```

```

    nit                VARCHAR(30) NOT NULL UNIQUE,
    correo_contacto    VARCHAR(150),
    hash_password      VARCHAR(255) NULL,          -- login propio del portal
    telefono           VARCHAR(30),
    latitud             DECIMAL(10,7),
    longitud            DECIMAL(10,7),
    score_actual        DECIMAL(5,2) NULL,          -- [4] rollup del score
    activo              TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB;

-- =====
-- M05: SOLICITUDES DE COMPRA
-- =====

CREATE TABLE solicitud_compra (
    id                  BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    empresa_id          BIGINT UNSIGNED NOT NULL,
    centro_costo_id     BIGINT UNSIGNED NOT NULL,
    usuario_solicitante_id BIGINT UNSIGNED NOT NULL,
    consecutivo          VARCHAR(30) NOT NULL UNIQUE,
    motivo              VARCHAR(500),
    fecha_requerida      DATE NOT NULL,
    estado              ENUM('BORRADOR', 'PENDIENTE_APROBACION', 'APROBADA',
                            'RECHAZADA', 'EN_RFQ', 'ADJUDICADA', 'CANCELADA', 'CERRADA')
                            NOT NULL DEFAULT 'BORRADOR',
    creado_en           DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (empresa_id) REFERENCES empresa(id),
    FOREIGN KEY (centro_costo_id) REFERENCES centro_costo(id),
    FOREIGN KEY (usuario_solicitante_id) REFERENCES usuario(id)
) ENGINE=InnoDB;

CREATE TABLE solicitud_compra_linea (
    id                  BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    solicitud_compra_id BIGINT UNSIGNED NOT NULL,
    producto_id         BIGINT UNSIGNED NOT NULL,
    cantidad            DECIMAL(18,4) NOT NULL,
    CHECK (cantidad > 0),
    FOREIGN KEY (solicitud_compra_id) REFERENCES solicitud_compra(id),
    FOREIGN KEY (producto_id) REFERENCES producto(id)
) ENGINE=InnoDB;

CREATE TABLE aprobacion_solicitud (
    id                  BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    solicitud_compra_id BIGINT UNSIGNED NOT NULL,
    usuario_aprobador_id BIGINT UNSIGNED NOT NULL,
    politica_aprobacion_id BIGINT UNSIGNED NOT NULL,
    decision            ENUM('APROBADA', 'RECHAZADA', 'DEVUELTA') NOT NULL,
    observaciones       VARCHAR(500),
    decidido_en         DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (solicitud_compra_id) REFERENCES solicitud_compra(id),
    FOREIGN KEY (usuario_aprobador_id) REFERENCES usuario(id),
    FOREIGN KEY (politica_aprobacion_id) REFERENCES politica_aprobacion(id)
) ENGINE=InnoDB;

-- =====
-- M06: RFQ Y COTIZACIONES
-- =====

```

```

CREATE TABLE rfq (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    solicitud_compra_id BIGINT UNSIGNED NOT NULL,
    usuario_id BIGINT UNSIGNED NOT NULL,
    consecutivo VARCHAR(30) NOT NULL UNIQUE,
    fecha_limite DATETIME NOT NULL,
    condiciones VARCHAR(1000),
    estado ENUM('ABIERTA', 'CERRADA', 'ADJUDICADA', 'CANCELADA') NOT NULL DEFAULT 'ABIERTA',
    FOREIGN KEY (solicitud_compra_id) REFERENCES solicitud_compra(id),
    FOREIGN KEY (usuario_id) REFERENCES usuario(id)
) ENGINE=InnoDB;

CREATE TABLE rfq_linea (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    rfq_id BIGINT UNSIGNED NOT NULL,
    producto_id BIGINT UNSIGNED NOT NULL,
    cantidad DECIMAL(18,4) NOT NULL,
    especificaciones VARCHAR(1000),
    FOREIGN KEY (rfq_id) REFERENCES rfq(id),
    FOREIGN KEY (producto_id) REFERENCES producto(id)
) ENGINE=InnoDB;

CREATE TABLE rfq_proveedor (
    rfq_id BIGINT UNSIGNED NOT NULL,
    proveedor_id BIGINT UNSIGNED NOT NULL,
    invitado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (rfq_id, proveedor_id),
    FOREIGN KEY (rfq_id) REFERENCES rfq(id),
    FOREIGN KEY (proveedor_id) REFERENCES proveedor(id)
) ENGINE=InnoDB;

CREATE TABLE cotizacion (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    rfq_id BIGINT UNSIGNED NOT NULL,
    proveedor_id BIGINT UNSIGNED NOT NULL,
    estado ENUM('BORRADOR', 'ENVIADA', 'EN_ANALISIS', 'SELECCIONADA',
        'NO_SELECCIONADA', 'RECHAZADA') NOT NULL DEFAULT 'BORRADOR',
    condiciones_comerciales VARCHAR(1000),
    documento_original_id BIGINT UNSIGNED NULL, -- FK a documento (archivo tal cual se recibió)
    creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (rfq_id) REFERENCES rfq(id),
    FOREIGN KEY (proveedor_id) REFERENCES proveedor(id)
) ENGINE=InnoDB;

CREATE TABLE cotizacion_linea (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    cotizacion_id BIGINT UNSIGNED NOT NULL,
    rfq_linea_id BIGINT UNSIGNED NOT NULL,
    producto_id BIGINT UNSIGNED NOT NULL,
    precio_unitario_ofertado DECIMAL(18,4) NOT NULL,
    unidad_ofertada VARCHAR(50) NOT NULL,
    descuento_pct DECIMAL(5,2) NOT NULL DEFAULT 0,
    confianza_extraccion DECIMAL(4,3) NULL, -- 0.000 - 1.000, dato de la IA
    FOREIGN KEY (cotizacion_id) REFERENCES cotizacion(id),
    FOREIGN KEY (rfq_linea_id) REFERENCES rfq_linea(id),
    FOREIGN KEY (producto_id) REFERENCES producto(id)
) ENGINE=InnoDB;

```



```

) ENGINE=InnoDB;

-- =====
-- M07: INTELIGENCIA Y ADJUDICACIÓN
-- =====

-- [3] cabecera del análisis: referencia la versión congelada de pesos
CREATE TABLE analisis_cotizacion (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    rfq_id BIGINT UNSIGNED NOT NULL,
    config_pesos_evaluacion_id BIGINT UNSIGNED NOT NULL, -- snapshot usado
    creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (rfq_id) REFERENCES rfq(id),
    FOREIGN KEY (config_pesos_evaluacion_id) REFERENCES config_pesos_evaluacion(id)
) ENGINE=InnoDB;

-- resultado agregado por cotización (para el ranking)
CREATE TABLE analisis_cotizacion_resultado (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    analisis_cotizacion_id BIGINT UNSIGNED NOT NULL,
    cotizacion_id BIGINT UNSIGNED NOT NULL,
    costo_total DECIMAL(18,2) NOT NULL,
    score_final DECIMAL(5,2) NOT NULL,
    posicion_ranking SMALLINT UNSIGNED NOT NULL,
    UNIQUE KEY uq_analisis_cotizacion (analisis_cotizacion_id, cotizacion_id),
    FOREIGN KEY (analisis_cotizacion_id) REFERENCES analisis_cotizacion(id),
    FOREIGN KEY (cotizacion_id) REFERENCES cotizacion(id)
) ENGINE=InnoDB;

-- [2] puntaje por criterio y por cotización (reemplaza las columnas fijas)
CREATE TABLE analisis_cotizacion_detalle (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    analisis_cotizacion_id BIGINT UNSIGNED NOT NULL,
    cotizacion_id BIGINT UNSIGNED NOT NULL,
    criterio_evaluacion_id BIGINT UNSIGNED NOT NULL,
    peso_aplicado DECIMAL(5,2) NOT NULL, -- copiado del snapshot
    puntaje DECIMAL(5,2) NOT NULL, -- 0-100
    score_ponderado DECIMAL(6,2) NOT NULL, -- puntaje * peso_aplicado / 100
    UNIQUE KEY uq_analisis_cot_criterio (analisis_cotizacion_id, cotizacion_id, criterio_evaluacion_id),
    FOREIGN KEY (analisis_cotizacion_id) REFERENCES analisis_cotizacion(id),
    FOREIGN KEY (cotizacion_id) REFERENCES cotizacion(id),
    FOREIGN KEY (criterio_evaluacion_id) REFERENCES criterio_evaluacion(id)
) ENGINE=InnoDB;

CREATE TABLE adjudicacion (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    rfq_id BIGINT UNSIGNED NOT NULL,
    analisis_cotizacion_id BIGINT UNSIGNED NOT NULL,
    cotizacion_seleccionada_id BIGINT UNSIGNED NOT NULL,
    usuario_id BIGINT UNSIGNED NOT NULL,
    sigue_recomendacion TINYINT(1) NOT NULL,
    justificacion VARCHAR(1000) NULL, -- obligatorio a nivel app si sigue_recomendacion = 0
    decidido_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (rfq_id) REFERENCES rfq(id),
    FOREIGN KEY (analisis_cotizacion_id) REFERENCES analisis_cotizacion(id),
    FOREIGN KEY (cotizacion_seleccionada_id) REFERENCES cotizacion(id),
    FOREIGN KEY (usuario_id) REFERENCES usuario(id)

```

```

) ENGINE=InnoDB;

-- =====
-- M08-M10: ÓRDENES, PAGO Y RECEPCIÓN
-- =====

CREATE TABLE orden_compra (
    id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    adjudicacion_id BIGINT UNSIGNED NOT NULL,
    proveedor_id      BIGINT UNSIGNED NOT NULL,
    consecutivo        VARCHAR(30) NOT NULL UNIQUE,
    total              DECIMAL(18,2) NOT NULL,
    lugar_entrega      VARCHAR(250) NOT NULL,
    version            INT UNSIGNED NOT NULL DEFAULT 1,
    estado             ENUM('EMITIDA', 'ACEPTADA_PROVEEDOR', 'EN_PREPARACION',
                           'DESPACHADA', 'RECIBIDA_PARCIAL', 'RECIBIDA_TOTAL',
                           'CERRADA', 'CANCELADA') NOT NULL DEFAULT 'EMITIDA',
    creado_en          DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (adjudicacion_id) REFERENCES adjudicacion(id),
    FOREIGN KEY (proveedor_id) REFERENCES proveedor(id)
) ENGINE=InnoDB;

CREATE TABLE orden_compra_linea (
    id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    orden_compra_id BIGINT UNSIGNED NOT NULL,
    producto_id       BIGINT UNSIGNED NOT NULL,
    cantidad          DECIMAL(18,4) NOT NULL,
    precio_unitario   DECIMAL(18,4) NOT NULL,
    subtotal          DECIMAL(18,2) NOT NULL,
    FOREIGN KEY (orden_compra_id) REFERENCES orden_compra(id),
    FOREIGN KEY (producto_id) REFERENCES producto(id)
) ENGINE=InnoDB;

-- [5] se agrega estado explícito (antes ausente en el MER)
CREATE TABLE pago (
    id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    orden_compra_id BIGINT UNSIGNED NOT NULL,
    metodo            VARCHAR(50) NOT NULL,
    valor             DECIMAL(18,2) NOT NULL,
    fecha_pago        DATE NOT NULL,
    referencia        VARCHAR(100),
    estado            ENUM('PENDIENTE', 'REGISTRADO') NOT NULL DEFAULT 'PENDIENTE',
    comprobante_documento_id BIGINT UNSIGNED NULL, -- FK a documento
    FOREIGN KEY (orden_compra_id) REFERENCES orden_compra(id)
) ENGINE=InnoDB;

CREATE TABLE recepcion (
    id                BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    orden_compra_id BIGINT UNSIGNED NOT NULL,
    usuario_id        BIGINT UNSIGNED NOT NULL,
    bodega_id         BIGINT UNSIGNED NOT NULL,
    recibido_en        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    observaciones      VARCHAR(500),
    FOREIGN KEY (orden_compra_id) REFERENCES orden_compra(id),
    FOREIGN KEY (usuario_id) REFERENCES usuario(id),
    FOREIGN KEY (bodega_id) REFERENCES bodega(id)
) ENGINE=InnoDB;

```

```

CREATE TABLE recepcion_linea (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  recepcion_id BIGINT UNSIGNED NOT NULL,
  orden_compra_linea_id BIGINT UNSIGNED NOT NULL,
  cantidad_recibida DECIMAL(18,4) NOT NULL,
  cantidad_faltante DECIMAL(18,4) NOT NULL DEFAULT 0,
  cantidad_rechazada DECIMAL(18,4) NOT NULL DEFAULT 0,
  FOREIGN KEY (recepcion_id) REFERENCES recepcion(id),
  FOREIGN KEY (orden_compra_linea_id) REFERENCES orden_compra_linea(id)
) ENGINE=InnoDB;

-- =====
-- EVALUACIÓN DE PROVEEDORES
-- =====

-- [4] evento por orden, con los 7 indicadores de la sección 18 de la guía
CREATE TABLE evaluacion_proveedor (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  proveedor_id BIGINT UNSIGNED NOT NULL,
  orden_compra_id BIGINT UNSIGNED NOT NULL,
  puntualidad DECIMAL(5,2) NULL,
  cumplimiento_cantidad DECIMAL(5,2) NULL,
  incidencias DECIMAL(5,2) NULL,
  calidad DECIMAL(5,2) NULL,
  precio DECIMAL(5,2) NULL,
  respuesta DECIMAL(5,2) NULL,
  score DECIMAL(5,2) NULL, -- ponderación configurable de los anteriores
  creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (proveedor_id) REFERENCES proveedor(id),
  FOREIGN KEY (orden_compra_id) REFERENCES orden_compra(id)
) ENGINE=InnoDB;

-- =====
-- AUDITORÍA, DOCUMENTOS Y NOTIFICACIONES
-- =====

-- [7] FK polimórfica real (entidad_tipo + entidad_id) en vez de
-- relaciones implícitas sin tipar
CREATE TABLE documento (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre_archivo VARCHAR(250) NOT NULL,
  ruta_storage VARCHAR(500) NOT NULL,
  tipo_mime VARCHAR(100) NOT NULL,
  tamano_bytes BIGINT UNSIGNED NOT NULL,
  entidad_tipo VARCHAR(50) NOT NULL, -- 'COTIZACION', 'ORDEN_COMPRA', 'RECEPCION', 'PAGO', 'PROVEEDOR',
etc.
  entidad_id BIGINT UNSIGNED NOT NULL,
  subido_por_usuario_id BIGINT UNSIGNED NULL,
  creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_documento_entidad (entidad_tipo, entidad_id),
  FOREIGN KEY (subido_por_usuario_id) REFERENCES usuario(id)
) ENGINE=InnoDB;

CREATE TABLE config_notificacion (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  evento VARCHAR(80) NOT NULL,

```

```

    canal VARCHAR(30) NOT NULL, -- EMAIL, INTERNA
    activo TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB;

CREATE TABLE notificacion (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    usuario_id BIGINT UNSIGNED NOT NULL,
    evento VARCHAR(80) NOT NULL,
    mensaje VARCHAR(500) NOT NULL,
    leida TINYINT(1) NOT NULL DEFAULT 0,
    creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (usuario_id) REFERENCES usuario(id)
) ENGINE=InnoDB;

CREATE TABLE auditoria (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    usuario_id BIGINT UNSIGNED NULL,
    entidad VARCHAR(80) NOT NULL,
    entidad_id BIGINT UNSIGNED NOT NULL,
    accion VARCHAR(80) NOT NULL,
    detalle TEXT NULL,
    creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_auditoria_entidad (entidad, entidad_id),
    FOREIGN KEY (usuario_id) REFERENCES usuario(id)
) ENGINE=InnoDB;

-- FKs diferidas que apuntan a documento (creada después por dependencia circular)
ALTER TABLE cotizacion ADD CONSTRAINT fk_cotizacion_documento
    FOREIGN KEY (documento_original_id) REFERENCES documento(id);
ALTER TABLE pago ADD CONSTRAINT fk_pago_documento
    FOREIGN KEY (comprobante_documento_id) REFERENCES documento(id);

SET FOREIGN_KEY_CHECKS = 1;

```